

Revision Notes for Class 12 History Chapter 6 – Searching

Searching is a key technique in computer science, used to find whether a particular value, called the key, is present in a collection and to determine its position. This process is important for retrieving data efficiently from storage. Understanding searching allows programmers to design faster and more effective algorithms for various real-world applications.

Sometimes you know exactly where your required item is, but often you need to search for it. Similarly, computers organize and search large amounts of data so users and programs can access information quickly. How the search is performed can affect how much time it takes to find data, especially as collections grow larger.

There are different ways to search for data. Each method has its own advantage depending on the data arrangement and the size of the list or collection. To perform efficient searches, it's important to select the most suitable algorithm depending on whether the data is sorted or not.

Linear Search

Linear search is the simplest search method and works by checking each element in the list one-after-another until the key is found or the entire list is exhausted. It is also called sequential or serial search. This method does not require the collection to be in any particular order.

- In linear search, the algorithm starts from the first element and moves towards the last, comparing each element with the target key.
- If a match is found, the position is returned and the search ends; otherwise, it continues until all elements are checked.
- Linear search is most suitable for small and unsorted lists, since it can take a long time on large collections.

For example, consider $\text{numList} = [8, -4, 7, 17, 0, 2, 19]$ and $\text{key} = 17$. Each value is checked in order, and after four comparisons the key is found at position 4. If the key is the first element, only one comparison is needed; if last, all elements are checked.

- If the key is not present, the algorithm checks every element and declares the search unsuccessful at the end.
- Linear search always does at least 1 comparison (if the key is first) and at most n comparisons (if not present or last).
- The method can be implemented easily in most programming languages, like Python, using a simple loop to traverse the list. The first matching position is usually returned.

Binary Search

Binary search is a much faster technique, but it only works when the list is sorted (in ascending or descending order). Rather than checking each item one by one, binary search cuts the search space in half with every step.

- The algorithm first checks the element in the middle of the list.
- If the middle element is equal to the key, the position is returned.
- If the key is less, the search continues on the left half; if greater, it continues on the right half.

With each comparison, binary search reduces the remaining search range by half, making it extremely efficient for large, sorted lists. For example, in the sorted list $\text{numList} = [2, 3, 5, 7, 10, 11, 12, 17, 19, 23, 29, 31, 37, 41, 43]$, searching for 17 starts at position 7 (middle) and finds the key in just one step.

- If the key is not present, binary search continues halving the list until the search range is empty, then returns unsuccessful.
- On average, binary search completes in $\log_2 n$ steps, making it much quicker than linear search on large lists.

Binary search must only be used on sorted data. If the list is not sorted, results will be incorrect. Common uses of binary search include looking up names in a telephone directory, searching dictionaries, and database indexing.

Search By Hashing

Hashing is another technique for fast searching. Here, a hash function computes an index from the key, allowing direct access in a table (hash table). This means, if no special cases occur, the key can be found in just one operation.

- A hash function, like $h(\text{element}) = \text{element} \% \text{table_size}$, calculates the index where the value should be stored or searched.
- If the table index is empty or contains the key, the search ends immediately.

For example, with the list $[34, 16, 2, 93, 80, 77, 51]$ and a table of size 10, keys are mapped as: $34 \rightarrow 4$, $16 \rightarrow 6$, $2 \rightarrow 2$, etc. If you want to search for 16, you check position 6 right away. This is very fast compared to checking each element sequentially.

However, sometimes two keys map to the same position. For example, both 16 and 26 produce a hash value of 6. This situation is called a collision. Special methods, called collision resolution techniques, are used to handle these situations.

- If each key gets its own unique position, the hash function is said to be perfect.
- Computing a hash index does not depend on the total number of elements, making hash search fast for large datasets.

Algorithm Overview and Example Codes

Each searching technique can be described using a simple algorithm. For linear search, a loop checks each element in order, stopping if the key is found.

For binary search, you keep updating the start and end positions and look at the middle value each time. In hashing, a hash function computes the specific slot to check directly.

- Linear search Python code uses a for loop to check each element; it returns the index (plus one) when found or None if not present.
- Binary search needs the list sorted and repeatedly finds the middle index until the key is found or the interval is empty.
- Hashing code applies the hash function to find the possible location immediately, reducing time compared to other methods.

Comparison of Searching Techniques

Linear search is easy to understand and use but becomes slow as the list grows. Binary search is much faster but only works on sorted data. Hashing is fastest for searching single items, provided collisions are rare or well-managed.

- Binary search is most efficient for large, sorted lists, often used in searching databases and directories.
- Hashing is powerful when used for quick lookups, as in dictionaries or when mapping data like countries to capitals.
- Choosing an appropriate search method depends on the data's nature and whether fast repeated lookup or list traversal is needed.

Key Points and Summary

To sum up, searching techniques are used to check if a key exists in a collection and, if present, find where it is. Linear search checks one by one, binary search halves the search space each time (on sorted lists), and hashing uses a special function for direct access.

- Linear search is simple but slow for large data.
- Binary search is very fast for sorted collections.
- Hash-based searching can be instant if there are no collisions.
- Collisions in hashing require extra steps, called collision resolution.
- In exams, remember: use linear search for small/unsorted lists, binary search for fast searches in sorted lists, and hashing for direct and quick lookup.

It is also useful to practice coding these methods and filling step tables for each approach. This helps in understanding not only how the algorithm works, but also in comparing their efficiency for exam questions.

Practice and Applications

Apply linear search to find the position of values in random or unordered lists. Use binary search for tasks like searching a sorted phonebook or list of roll numbers. Hashing is perfect for creating dictionaries where quick lookup by key, such as country-capital pairs, is important.

- Practicing with code and activity tables from this chapter will help cement understanding and prepare for exam scenarios.
- Try building small programs that use each technique.

Understanding these differences and uses of each searching method will not only help you in board exams but also build a strong foundation for studying data structures and optimization in higher studies.

